

Taller: Análisis y Aplicación de Principios SOLID

*Diagnóstico de violación del SRP y diseño OCP para
GestorTransaccionPrincipal*

Jhordan García
Emmanuel Ospina
Ingeniería en Sistemas
Fundamentos de Software – Semana 3
2026

1. Código Fuente – codigo_fragil_srp.py

A continuación se presenta el código entregado por el equipo de auditoría Quantum Core para su análisis. La clase `GestorTransaccionPrincipal` concentra en un único método, `procesar_y_validar_y_reportar()`, tres tareas de naturaleza distinta: validar los datos de entrada, calcular el impacto de la transacción y generar el reporte en texto plano.

```
# codigo_fragil_srp.py
# Semana 3 - Actividad 2: codigo para ANALIZAR (no hay que corregirlo en codigo,
# solo diagnosticar las violaciones de SRP y OCP en el informe).

class GestorTransaccionPrincipal:
    def __init__(self, datos):
        self.datos = datos # ej: {"id": "T001", "tipo": "CREDITO", "monto": 500000}

    def procesar_y_validar_y_reportar(self):
        # 1) RESPONSABILIDAD: validacion de datos
        if self.datos["monto"] < 0:
            print("ERROR: el monto no puede ser negativo.")
            return

        if self.datos["tipo"] not in ("CREDITO", "DEBITO"):
            print("ERROR: tipo de transaccion desconocido.")
            return

        # 2) RESPONSABILIDAD: logica de negocio
        if self.datos["tipo"] == "CREDITO":
            impacto = self.datos["monto"] * 0.02
        else:
            impacto = 1500

        # 3) RESPONSABILIDAD: reporte (formato texto plano)
        print("===== REPORTE =====")
        print("ID:", self.datos["id"])
        print("Tipo:", self.datos["tipo"])
        print("Monto:", self.datos["monto"])
        print("Impacto:", impacto)

# Ejemplo de uso
gestor = GestorTransaccionPrincipal({"id": "T001", "tipo": "CREDITO", "monto": 500000})
gestor.procesar_y_validar_y_reportar()
```

2. Paso A – Violación del Principio de Responsabilidad Única (SRP)

2.1 Diagnóstico

El método `procesar_y_validar_y_reportar()` viola el SRP porque un mismo bloque de código tiene simultáneamente tres motivos distintos para cambiar, cuando el principio establece que una clase o método debería tener una única razón para ser modificado. Las tres responsabilidades que mezcla, marcadas incluso por el propio autor del código en los comentarios, son:

- Validación de datos: comprueba que el monto no sea negativo y que el tipo de transacción sea CREDITO o DEBITO, imprimiendo un error y retornando si algo es inválido. Es una responsabilidad de integridad de datos, ajena al cálculo.
- Lógica de negocio: calcula el impacto de la transacción (2% del monto si es CREDITO, o un valor fijo de 1500 si es DEBITO). Esta es la regla de negocio real que determina el resultado financiero.

- Generación y presentación del reporte: imprime en consola, línea por línea y en texto plano, el ID, tipo, monto e impacto de la transacción. Es una responsabilidad de formato y salida, completamente distinta a validar o calcular.

El riesgo concreto es que un cambio en cualquiera de estas tres áreas —por ejemplo, una nueva regla de validación, un ajuste en el porcentaje de impacto, o un cambio en cómo se presenta el reporte— obliga a modificar el mismo método, aumentando la probabilidad de introducir errores en las otras dos funcionalidades que no debían tocarse.

2.2 Propuesta de solución

Para aislar estas responsabilidades se propone dividir la clase original en tres nuevas clases, cada una con una única razón para cambiar:

- ValidadorDatos: encargada exclusivamente de recibir los datos crudos de la transacción y verificar su validez (monto no negativo, tipo reconocido). Su único trabajo es decir si los datos son correctos o no.
- MotorLogica: encargada exclusivamente de aplicar la regla de negocio sobre datos ya validados (calcular el impacto según el tipo de transacción). No conoce nada sobre validación ni sobre cómo se presenta el resultado.
- GeneradorReporte: encargada exclusivamente de tomar el resultado ya calculado por el motor de lógica y transformarlo en una salida presentable (por ahora texto plano). No conoce las reglas de negocio ni las reglas de validación.

Esta división facilita el mantenimiento futuro porque cada clase puede probarse de forma aislada e independiente, reduciendo el riesgo de efectos secundarios al hacer cambios; distintos integrantes del equipo pueden trabajar en paralelo sobre cada clase sin generar conflictos; y cuando un requisito cambie, el impacto queda contenido en una sola clase, sin arriesgar las otras dos funcionalidades, que es exactamente el problema que hoy tiene GestorTransaccionPrincipal.

3. Paso B – Principio Abierto/Cerrado (OCP)

3.1 Análisis del riesgo

Si el equipo de marketing exige que el reporte pase de texto plano a XML o JSON, y esta modificación se realiza directamente dentro de GestorTransaccionPrincipal, se estaría violando el Principio Abierto/Cerrado (OCP), que establece que una clase debe estar abierta para extensión pero cerrada para modificación. En el diseño actual, como las tres responsabilidades conviven en el mismo método, no es posible tocar únicamente la parte del reporte: el desarrollador debe abrir y editar el mismo bloque de código donde también viven la validación y el cálculo del impacto.

Esto representa un riesgo sistémico porque cualquier error introducido al escribir el nuevo formato XML o JSON puede romper, sin intención alguna, la validación de datos o el cálculo del impacto, funcionalidades que ya estaban probadas y funcionando correctamente. Un cambio pensado únicamente para el área de presentación termina poniendo en riesgo la integridad financiera del sistema, simplemente porque el diseño no separó las responsabilidades desde el inicio.

3.2 Solución OCP conceptual

El uso de Polimorfismo y Herencia resuelve el problema del OCP porque permite agregar nuevos formatos de reporte creando clases nuevas, en lugar de modificar la clase de lógica de negocio ya existente. La jerarquía propuesta sería:

- ReporteBase: clase base abstracta que define el contrato común, por ejemplo un método generar(datos_resultado), que toda clase hija está obligada a implementar. MotorLogica solo conoce y depende de este contrato, nunca de una implementación concreta.
- ReporteTextoPlano: hereda de ReporteBase e implementa generar() devolviendo el formato de texto que ya existe hoy, preservando el comportamiento actual.
- ReporteXML: hereda de ReporteBase e implementa generar() devolviendo el resultado envuelto en etiquetas XML.
- ReporteJSON: hereda de ReporteBase e implementa generar() devolviendo el resultado como un objeto JSON serializado.

Gracias al polimorfismo, quien orquesta el proceso simplemente recibe un objeto de tipo ReporteBase y le llama generar(datos_resultado), sin necesidad de saber si por dentro es texto plano, XML o JSON. Cuando marketing pida un nuevo formato, basta con crear una nueva clase hija que herede de ReporteBase e implemente su propio generar(), sin abrir ni modificar una sola línea de la clase de lógica de negocio original.

Este diseño cumple con estar abierto para extensión, porque el sistema admite nuevos formatos simplemente añadiendo clases nuevas, y cerrado para modificación, porque ni GestorTransaccionPrincipal ni MotorLogica ni las clases de reporte ya existentes y probadas necesitan ser tocadas para que el nuevo formato funcione. Esto elimina el riesgo sistémico descrito en el punto anterior, ya que la validación y la lógica de negocio quedan completamente aisladas de cualquier cambio futuro en la forma de presentar los resultados.

4. Conclusión

El caso de GestorTransaccionPrincipal evidencia cómo la falta de separación de responsabilidades no solo afecta la legibilidad del código, sino que compromete directamente la estabilidad del sistema ante cualquier cambio futuro. Aplicar SRP mediante la creación de ValidadorDatos, MotorLogica y GeneradorReporte, y complementarlo con OCP mediante una jerarquía polimórfica de reportes (ReporteBase, ReporteTextoPlano, ReporteXML, ReporteJSON), permite que el sistema crezca de forma segura: se agregan nuevas capacidades creando código nuevo, sin arriesgar el código de negocio ya validado y en producción.